NASCOMSOFT EMBEDDED LTD

# Course Registration & Enrollment Management System

Project outline · scope · objectives · implementation roadmap

An instructor-ready FastAPI mini-project plan for beginner backend developers. Designed for a 2–3-hour guided workshop, with a clear feature scope, a realistic student-registration workflow, and a path from local development to GitHub and deployment.

| | |
|---|---|
| **PROJECT TYPE** | Practical FastAPI workshop application |
| **PRIMARY USER** | Nascomsoft training administrator |
| **BACKEND** | FastAPI, in-memory Python data |
| **FRONTEND** | Simple HTML interface and forms |
| **CORE LEARNING** | CRUD, API endpoints, validation, Git/GitHub, deployment |
| **DELIVERY TARGET** | Local demonstration first; public deployment if time permits |

# 1. Project Overview

The **Nascomsoft Course Registration & Enrollment Management System** is a small web application that helps Nascomsoft Embedded LTD organize training courses and student enrollments. Instead of relying only on notebooks, scattered spreadsheets, or messages, training staff can register students, view courses, enroll students, and check who belongs to each course.

This is an educational prototype inspired by a training-center workflow—not a claim about Nascomsoft's existing internal operations.

### Core user journey

| | |
|---|---|
| **1. Web interface** | A staff member opens the app and completes a registration form. |
| **2. HTTP request** | The form sends the registration information to the backend. |
| **3. FastAPI backend** | The backend checks required fields and business rules. |
| **4. In-memory storage** | Python lists and dictionaries hold sample courses and enrollments. |
| **5. HTTP response** | The app confirms the registration or explains an error. |

**Important:** Data stored in memory may disappear when the application restarts or is redeployed. This is acceptable for classroom demonstrations, not for real student records.

# 2. Project Objectives

### A. Software objectives

- View available Nascomsoft training courses.
- Register students for a selected course.
- View all registered students.
- View students enrolled in a particular course.
- Find an enrollment using its ID.
- Update registration details.
- Remove an enrollment when necessary.
- Receive clear success and error messages.

### B. Learning objectives

- Translate requirements into features and a realistic project scope.
- Create and organize a FastAPI backend.
- Apply HTTP methods to actual business operations.
- Implement CRUD on enrollment records using temporary Python storage.
- Use path and query parameters correctly.

- Connect a simple frontend to backend behavior.
- Handle invalid requests and missing records.
- Test endpoints using FastAPI interactive API documentation.
- Use Git and GitHub to publish the source code and deploy a demonstration application.

**Main learning outcome:** Students can explain the complete journey from entering data in a browser to processing it in FastAPI and returning a response.

## 3. Who Will Use the System?

**Primary user: Nascomsoft Training Administrator.** This user registers students, selects courses, views enrollments, corrects details, and removes incorrect records.

The student is the person being registered, **not** an application user who needs to log in. Separate student/admin accounts would require authentication and permission management, which are excluded from the workshop.

## 4. Functional Scope: Three Modules

| Module | Required behavior |
|---|---|
| Course Catalog | Show available courses, course names, duration, and short descriptions. Preload course records; no course-creation screen required. |
| Student Registration | Capture student information, assign one course, and check essential fields before accepting a new registration. |
| Enrollment Management | Display all registrations, filter by course, retrieve one by ID, update details, and delete a registration. |

### Illustrative training courses

| Course ID | Course |
|---|---|
| 1 | Python Programming |
| 2 | Software Development with Python |
| 3 | Embedded Systems Fundamentals |
| 4 | Web Development |
| 5 | Data Analysis with Python |

These are illustrative workshop entries, not a verified catalog of current courses.

# 5. Minimum Viable Product (MVP)

| Essential: Build Today | Optional: If Time Permits |
|---|---|
| View available courses | Create or edit courses |
| Register a student | Search by student name |
| View all registrations | Show total registration count |
| View registration by ID | Display course enrollment counts |
| Filter by course | Add registration dates |
| Update registration | Confirm before deleting |
| Delete registration | Improve styling or export data |

**Scope decision:** Treat each student registration as one enrollment record linked to one course. Avoid complicated student accounts, multiple simultaneous enrollments, course capacities, or timetable conflicts.

### Not included in the workshop

Payment processing, accounts/login, JWT, PostgreSQL, SQLAlchemy, Pydantic-based schemas, email notifications, certificates, file uploads, and advanced frontend frameworks are explicitly out of scope.

# 6. Application Interface and User Experience

The interface should be functional and beginner-friendly. The focus is FastAPI, not advanced HTML/CSS. Prefer one dashboard page over complex navigation.

| Interface Area | What It Shows |
|---|---|
| Registration form | Name, email, phone number, course selection, Register action, and success/error feedback. |
| Enrollment dashboard | Table containing enrollment ID, student name, course, email, and available actions such as Edit/Delete. |
| Available courses | Course names and brief descriptions; ideally on the main page. |

**Implementation note:** Basic HTML forms can submit directly to FastAPI. Native HTML forms support GET and POST, but not PUT or DELETE; those API methods can still be implemented and tested through **/docs**, while browser editing/deleting can use form-handling routes if added.

# 7. Project Data Model

## Course record

| Field | Example | Purpose |
|---|---|---|
| course_id | 2 | Unique course identifier |
| title | Software Development with Python | Course name |
| duration | 4 Weeks | Course duration |
| description | Python backend development | Brief course description |

## Enrollment record

| Field | Example | Purpose |
|---|---|---|
| enrollment_id | 101 | Unique registration ID |
| student_name | Ada James | Student name |
| email | ada@example.com | Contact address |
| phone | 08012345678 | Phone number |
| course_id | 2 | References selected course |

**Relationship:** One course can have many enrollment records. Each workshop enrollment record refers to one course using its course ID.

# 8. Backend API Design

## Course endpoints

| Method | Endpoint | Purpose |
|---|---|---|
| GET | /courses | List available courses |
| GET | /courses/{course_id} | Get a single course |

## Enrollment endpoints

| Method | Endpoint | Purpose |
|---|---|---|
| GET | /enrollments | List all registrations |
| GET | /enrollments/{enrollment_id} | Retrieve one enrollment |
| GET | /enrollments?course_id=2 | Filter by selected course |
| POST | /enrollments | Register a new student |
| PUT | /enrollments/{enrollment_id} | Update an enrollment |
| DELETE | /enrollments/{enrollment_id} | Delete an enrollment |

## CRUD mapping

| CRUD action | Meaning | HTTP method |
|---|---|---|
| Create | Add an enrollment | POST |
| Read | Retrieve one/all enrollments | GET |
| Update | Change existing registration | PUT |
| Delete | Remove a registration | DELETE |

## Expected response behavior

New valid enrollment: **201 Created**. Successful retrieval: **200 OK**. Successful deletion: **200 OK** with confirmation, or **204 No Content** without a response body. Missing enrollment: **404 Not Found**. Invalid input: an appropriate **4xx** error with an understandable message.

## 9. Business Rules

| Condition | Expected behavior |
|---|---|
| Empty student name | Reject registration |
| Missing email | Reject registration |
| No course chosen | Reject registration |
| Unknown course ID | Reject registration |
| Missing enrollment ID | Return 404 Not Found |
| Valid registration | Create unique enrollment ID |
| Update existing enrollment | Preserve original enrollment ID |
| Delete existing enrollment | Record no longer appears |
| Multiple registrations | Assign distinct IDs |

Optional practice: Prevent duplicate registration using the same email and course combination. This introduces extra lookup logic and should not block the MVP.

## 10. Detailed Implementation Roadmap

### Phase 1 — Understand and plan | 15 min

**Work:** Explain the problem, identify the administrator, agree on MVP features, and draw the user journey.

**Output:** A clear feature scope and registration workflow.

### Phase 2 — Set up project | 15 min

**Work:** Create the directory and virtual environment; confirm FastAPI/Uvicorn; prepare files and verify server startup.

**Output:** Working FastAPI starter application.

### Phase 3 — Prepare in-memory data | 15 min

**Work:** Define sample courses, enrollment storage, fields, relationships, and unique ID approach.

**Output:** Consistent temporary data model.

### Phase 4 — Build read-only API | 20 min

**Work:** List courses and enrollments; get one enrollment; optionally filter by course; test in /docs.

**Output:** Working GET endpoints.

### Phase 5 — Implement CRUD | 35 min

**Work:** Register, read, update, and delete enrollments; handle invalid fields and missing records.

**Output:** Working CRUD backend with clear responses.

### Phase 6 — Create simple frontend | 30 min

**Work:** Add branded heading, course selector, registration form, enrollment table, and feedback.

**Output:** Browser can create and display registrations.

### Phase 7 — Test and debug | 15 min

**Work:** Check happy paths, errors, updates, deletes, and refresh behavior; fix issues.

**Output:** Reliable local demonstration.

### Phase 8 — Git and GitHub | 15 min

**Work:** Review changes, write useful commits, update README/dependencies, configure .gitignore, push project.

**Output:** Documented repository and commit history.

### Phase 9 — Deploy and demonstrate | 20 min

**Work:** Connect repository to Python hosting platform; deploy app; test public site and endpoint.

**Output:** Public demonstration, if hosting permits.

**Total suggested time:** 180 minutes. This is ambitious for beginners. If installation or hosting takes longer than expected, completing and testing the local FastAPI application is the priority.

## 11. GitHub and Deployment Strategy

| | |
|---|---|
| **Local project** | Implement and test FastAPI and the HTML interface. |
| **GitHub repository** | Track code changes, maintain README and dependency file, push source. |
| **Python hosting** | Deploy the running FastAPI application using a service such as Render. |
| **Public demo URL** | Open the hosted app and demonstrate a request/registration. |

GitHub is the source-code host, **not** the running Python server. GitHub Pages does not execute server-side FastAPI applications. A separate Python-compatible hosting service is required.

**Data safety:** Do not use actual student records in this unauthenticated, in-memory classroom prototype. Restarts or redeployments can clear data; open public endpoints are not appropriate for real student personal information.

## 12. Testing and Acceptance Criteria

| Test scenario | Expected result |
|---|---|
| Open homepage | Main page loads |
| View courses | Sample course records appear |
| Register valid student | New enrollment created |
| View enrollments | New record appears |
| Get existing ID | Correct record returned |
| Get missing ID | 404 Not Found |
| Filter by course | Matching registrations only |
| Update registration | Changes can be retrieved |
| Delete registration | Record no longer appears |
| Submit missing required fields | Request rejected |
| Refresh page | Records remain while same process is running |
| Restart app | In-memory records may be lost |
| Open public deployment | Application responds successfully |

### Completion checklist

- FastAPI starts and courses are available.
- Enrollment CRUD endpoints operate correctly.
- A basic HTML form can create an enrollment.
- The enrollment list displays records.
- Invalid input and missing IDs are handled.
- The endpoints are tested in /docs.
- README, dependencies, .gitignore, and GitHub history are present.
- Local or public demonstration is completed.

## 13. Student Deliverables

Students submit a GitHub repository containing the application, README, dependency information, and meaningful commits. If deployment succeeds, they also provide the public URL and a short demonstration of the registration process.

Their demonstration must answer: **(1)** What problem is solved? **(2)** How does the form communicate with FastAPI and back? **(3)** Where are Create, Read, Update and Delete implemented?

### Assessment rubric

| Assessment area | Marks |
|---|---|
| Problem understanding and scope | 10 |
| FastAPI route and endpoint design | 15 |
| CRUD functionality | 25 |
| Frontend integration | 15 |
| Validation and errors | 10 |
| Testing and debugging | 10 |
| Git/GitHub and README | 10 |
| Demonstration and explanation | 5 |
| **Total** | **100** |

## 14. Future Development Opportunities

| Future Version | Next Capabilities |
|---|---|
| Version 2 — Permanent storage | Replace in-memory records with PostgreSQL so enrollments survive restarts. |
| Version 3 — Authentication | Add administrator login, secure password handling, and authorization. |
| Version 4 — Training management | Course capacity limits, enrollment statistics, attendance and reporting. |

## 15. Final Project Direction

The project should demonstrate a complete, realistic software development cycle without introducing unnecessary technologies. Students will plan the requirements, build the FastAPI backend, connect a simple interface, test CRUD, publish their work to GitHub, and deploy if feasible.

**Recommended priority:** Build and test the backend first, integrate the simple web interface second, and deploy only when the core workflow is functioning. The purpose is to apply existing Python and FastAPI knowledge to a complete project.